GMS-KVCache System

Architecture & Implementation Report

GFS-Inspired Distributed KV Cache Management for Fault-Tolerant LLM Inference

University of Colorado Boulder | April 2026

1. Project Overview

This document describes the architecture and implementation plan for a GFS-inspired distributed KV cache management system. The core motivation is straightforward: in a multi-vLLM deployment under PD-disaggregated or chunked-prefill inference, when a worker process fails, its GPU is temporarily removed from service. Without persistent cache, all KV blocks held by that worker are lost and must be recomputed when the GPU is reinstated.

Our system trades MinIO disk space for faster worker recovery time. By persisting KV cache blocks to a shared MinIO backend and maintaining a Global Metadata Service (GMS) that tracks block ownership and location, a failed worker can reload its hot cache from MinIO upon restart, rather than waiting for cache to be rebuilt through organic inference traffic.

1.1 Core Trade-off

Dimension	Without GMS	With GMS
Worker recovery	Full cache rebuild via organic traffic	Hot cache reloaded from MinIO on restart
Storage cost	Zero remote storage	MinIO disk space proportional to hot cache size
TTFT after recovery	High (cold cache)	Near-baseline (warm cache restored)
Metadata overhead	None	GMS RPC latency (~5ms per lookup)

2. System Architecture

The architecture follows GFS design principles: the Global Metadata Service handles only metadata (block locations, ownership, admission decisions), while KV bytes flow directly between vLLM workers and MinIO nodes via the S3 API. The GMS never touches KV data.

2.1 High-Level Architecture

```
┌──────────────────────────────────────────┐
│          Global Metadata Service         │
│  ┌──────────────┐  ┌──────────────────┐  │
│  │   Block      │  │   Ownership      │  │
│  │  Location    │  │   Registry       │  │
│  │   Index      │  │   (epoch/lease)  │  │
│  └──────────────┘  └──────────────────┘  │
│  ┌──────────────┐  ┌──────────────────┐  │
│  │ Admission    │  │   Head Registry  │  │
│  │   Policy     │  │   (heartbeats)   │  │
```

```
                    │  └──────────┘   └────────────────┘   │
                    │         Raft Replicated              │
                    └──────────────────────────────────────┘
                                   │ control plane (gRPC)
          ┌────────────────────────┼────────────────────────┐
          │                        │                        │
   ┌──────▼──────┐          ┌──────▼──────┐          ┌──────▼──────┐
   │  vLLM #1    │          │  vLLM #2    │          │  vLLM #3    │
   │  Prefill    │          │  Decode     │          │  Decode     │
   │ ┌─────────┐ │          │ ┌─────────┐ │          │ ┌─────────┐ │
   │ │Metadata │ │          │ │Metadata │ │          │ │Metadata │ │
   │ │ Client  │ │          │ │ Client  │ │          │ │ Client  │ │
   │ └─────────┘ │          │ └─────────┘ │          │ └─────────┘ │
   └─────────────┘          └─────────────┘          └─────────────┘
          │                        │                        │
          └────────────────────────┼────────────────────────┘
                                   │
                   data plane (direct S3 API)
                                   │
          ┌────────────────────────┼────────────────────────┐
          │                        │                        │
   ┌──────▼──────┐          ┌──────▼──────┐          ┌──────▼──────┐
   │  MinIO #1   │          │  MinIO #2   │          │  MinIO #3   │
   └─────────────┘          └─────────────┘          └─────────────┘
```

2.2 Component Roles

Component	Role	Key Responsibility
Global Metadata Service (GMS)	Control plane	Block location index, ownership tracking, epoch management, admission decisions
MetadataClient	Per-vLLM library	Register blocks, query locations, report heartbeats, receive prefetch hints
MinIO Cluster	Data plane	Persistent KV block storage via S3 API; never touched by GMS
vLLM Workers	Inference	Prefill / decode; read/write KV blocks directly to MinIO; register with GMS

3. Global Metadata Service Design

3.1 Block Metadata Schema

Each KV cache block is identified by a content-addressed chunk hash (LMCache's rolling prefix hash). The GMS maintains the following per-block record:

```
@dataclass
class BlockMetadata:
```

```
    # Identity
    chunk_hash:        int        # LMCache rolling prefix hash (content-addressed)

    # Location
    minio_node:        str        # Which MinIO node holds this block
    bucket:            str        # S3 bucket name
    object_key:        str        # S3 object key
    size_bytes:        int        # Block size in bytes

    # Ownership (fault tolerance)
    owner_worker_id:   str        # vLLM instance currently owning this sequence
    epoch:             int        # Monotonically increasing; incremented on
ownership transfer
    state:             str        # PENDING | COMMITTED

    # Access tracking
    created_at:        float      # Unix timestamp
    last_accessed:     float      # For LRU eviction and recency scoring
    access_count:      int        # For frequency-based admission

    # Deduplication
    refcount:          int        # How many workers share this block
```

3.2 Ownership and Epoch Fencing

Ownership is tracked per-sequence, not per-block. A sequence is a single ongoing decode request. Only one worker may write to a sequence's KV cache at any time. When a worker fails, the GMS transfers ownership to a new worker and increments the epoch.

Epoch fencing prevents stale writes from a recovered-but-evicted worker corrupting MinIO:

```
# Ownership transfer on worker failure
def transfer_ownership(seq_id: str, new_worker_id: str):
    entry = ownership_registry[seq_id]
    entry.epoch += 1                    # Increment epoch
    entry.owner_worker_id = new_worker_id
    entry.state = 'ACTIVE'

# Worker must present its epoch on every write registration
def register_write(chunk_hash: int, seq_id: str, worker_epoch: int) -> bool:
    current_epoch = ownership_registry[seq_id].epoch
    if worker_epoch < current_epoch:
        return False  # Reject: stale epoch (old worker trying to write after
eviction)
    # Proceed: mark block as PENDING
    block_index[chunk_hash].state = 'PENDING'
    return True

# Worker confirms write success
def commit_write(chunk_hash: int, seq_id: str, worker_epoch: int) -> bool:
    if worker_epoch < ownership_registry[seq_id].epoch:
```

```
        return False  # Reject: stale epoch
    block_index[chunk_hash].state = 'COMMITTED'
    return True
```

3.3 Two-Phase Commit Protocol

To handle write uncertainty (worker crashes after writing to MinIO but before notifying GMS), every write follows a two-phase protocol:

Phase	Action	Block State	Safe to Read?
Phase 1: Pre-write	Worker calls register_write() on GMS	PENDING	No
Phase 2: Confirm	Worker calls commit_write() after S3 PUT succeeds	COMMITTED	Yes
Recovery	New owner sees PENDING blocks, discards them, recomputes	Discarded	N/A

This guarantees that a new owner after failover only reads COMMITTED blocks, which are guaranteed to exist in MinIO. PENDING blocks are treated as lost and recomputed from the committed frontier.

3.4 Committed Frontier

The GMS maintains a committed_frontier per sequence: the index of the last COMMITTED block. During failover:

- Blocks at index <= committed_frontier: safe to load from MinIO
- Blocks at index > committed_frontier: may be PENDING or missing; discard and recompute
- At most one uncommitted block exists at any time (invariant enforced by the write protocol)

This bounds recovery cost: the new owner recomputes at most one block, regardless of how long the sequence had been running.

3.5 Failure Detection

The GMS detects worker failures via heartbeat timeouts:

```
HEARTBEAT_INTERVAL = 5   # seconds; workers send heartbeat every 5s
```

```
FAILURE_THRESHOLD  = 2   # missed heartbeats before declaring failure
FAILURE_TIMEOUT    = 10  # seconds until GMS initiates ownership transfer

# On timeout: GMS transfers ownership of all sequences owned by the failed worker
# New owner is selected from the pool of healthy workers
```

3.6 Content-Addressed Deduplication

LMCache computes chunk hashes as a rolling prefix hash: h_i = hash(h_{i-1}, tokens_i). Because this is deterministic and content-addressed, two workers processing the same prompt prefix produce identical chunk hashes. The GMS exploits this:

- Before writing a block, the worker queries GMS: does chunk_hash X exist?
- If yes: skip the S3 PUT; GMS increments refcount. Zero extra storage cost.
- If no: write to MinIO; GMS registers the new block.

CRITICAL deployment requirement: all vLLM instances must set PYTHONHASHSEED to the same fixed value. Python's built-in hash() is randomized per process by default, which breaks cross-instance deduplication.

```
# In deployment script or docker-compose.yml
export PYTHONHASHSEED=42
```

3.7 Admission Policy

Not every KV block is worth persisting to MinIO. The GMS gates remote writes using a cost-model inequality:

```
# Admit to MinIO only if:
C_fetch < C_recompute * P_reuse

# Where:
C_fetch     = EWMA of MinIO round-trip latency for this block size (~0.55s
baseline)
C_recompute = estimated prefill time for this chunk (~1.60s baseline for Mistral-7B
C256)
P_reuse     = hybrid reuse probability estimate (see below)

# Break-even: P_reuse > 0.55 / 1.60 = 0.344
# Blocks with P_reuse < 0.344 are not worth writing to MinIO
```

Reuse probability is estimated from three signals with configurable weights:

Signal	Formula	Weight (default)
Frequency	min(1, f / f_thresh) where f is Count-Min Sketch estimate	0.4
Workload class	Shared prefix: 0.85 | RAG: 0.75 | Multi-turn: 0.65 | Unique: 0.10	0.4
Recency	exp(-lambda * (t_now - t_last)) with lambda=0.1	0.2

4. MetadataClient Design

The MetadataClient is a lightweight library embedded in each vLLM instance. It replaces direct MinIO existence checks with GMS queries and intercepts the LMCache write path to enforce admission and register blocks.

4.1 Client API

```
class MetadataClient:
    def lookup(self, chunk_hashes: List[int]) -> LookupResult:
        """Query GMS for block locations. Returns hits + prefetch hints."""

    def check_admit(self, chunk_hash: int, size_bytes: int) -> AdmitDecision:
        """Ask GMS whether this block should be written to MinIO."""

    def register_write(self, chunk_hash: int, seq_id: str, epoch: int) -> bool:
        """Phase 1 of 2PC: declare intent to write. Returns False if epoch
stale."""

    def commit_write(self, chunk_hash: int, seq_id: str, epoch: int,
                     minio_node: str, object_key: str) -> bool:
        """Phase 2 of 2PC: confirm write success. Returns False if epoch stale."""

    def report_access(self, chunk_hashes: List[int]) -> None:
        """Batched async access reporting (every 100ms window)."""

    def heartbeat(self) -> None:
        """Called every 5s to signal liveness to GMS."""
```

4.2 Write Path (with Admission + 2PC)

```
async def write_block(chunk_hash, tokens, seq_id, epoch):
    # Step 1: Deduplication check
    if await client.lookup([chunk_hash]).is_hit:
        return  # Block already in MinIO, skip write

    # Step 2: Admission gate
```

```
    decision = await client.check_admit(chunk_hash, size_bytes)
    if decision == SKIP:
        return  # Not worth writing to MinIO

    # Step 3: Phase 1 - register intent
    ok = await client.register_write(chunk_hash, seq_id, epoch)
    if not ok:
        return  # Epoch stale; this worker lost ownership

    # Step 4: Write to MinIO directly (S3 PUT)
    minio_node, object_key = decision.target_node, make_key(chunk_hash)
    await s3_client.put_object(minio_node, object_key, kv_tensor_bytes)

    # Step 5: Phase 2 - confirm success
    await client.commit_write(chunk_hash, seq_id, epoch, minio_node, object_key)
```

4.3 Read Path (Failover Recovery)

```
async def recover_sequence(seq_id, new_epoch):
    # GMS returns committed_frontier and list of COMMITTED block locations
    frontier, committed_blocks = await client.get_committed_blocks(seq_id)

    # Load all committed blocks from MinIO
    for block in committed_blocks:  # These are guaranteed to exist in MinIO
        kv_data = await s3_client.get_object(block.minio_node, block.object_key)
        load_into_gpu_cache(kv_data)

    # Blocks after frontier: recompute (at most 1 block by invariant)
    recompute_from_token(frontier.last_committed_token + 1)
```

5. Data Plane: MinIO & S3

KV bytes flow directly between vLLM workers and MinIO via the standard S3 API. The GMS is never on the data path.

5.1 Object Naming Convention

```
# Object key format: {chunk_hash_hex}
# Example: kvcache/ab12cd34ef567890

bucket = 'kvcache'
object_key = f'{chunk_hash:016x}'

# MinIO node selection: jump consistent hash on chunk_hash
# This ensures the same block always maps to the same MinIO node
# (unless GMS overrides for load balancing)
```

```
target_node = jump_hash(chunk_hash, num_minio_nodes)
```

5.2 MinIO Node Assignment

Jump consistent hash maps each chunk_hash to a primary MinIO node. Jump hashing provides minimal disruption on node add/remove (only 1/n of keys migrate) and runs in O(log n) time. The GMS records the actual placement and returns it on lookup, so workers always write to and read from the correct node.

5.3 Deployment Ports

Service	Port	Protocol
GMS (primary)	8100	gRPC
GMS (Raft peer 1)	8101	gRPC
GMS (Raft peer 2)	8102	gRPC
MinIO API	9000	S3 / HTTP
MinIO Console	9001	HTTP
vLLM instance 1	8000	HTTP
vLLM instance 2	8001	HTTP
LMCache Controller	8200	ZMQ / gRPC

6. Fault Tolerance Model

6.1 Failure Taxonomy

Failure Type	Detection	Recovery Action	In Scope?
Worker crash	Heartbeat timeout (10s)	Ownership transfer; new owner loads from committed_frontier	YES
Worker slow / partial failure	Heartbeat timeout	Same as crash	YES
Write uncertainty	PENDING blocks on recovery	New owner discards PENDING, recomputes	YES
Split brain (old worker revives)	Epoch fencing at GMS	Stale epoch rejected; old worker stops	YES
MinIO node down	S3 API error	Blocks on that node unavailable; recompute (MinIO HA is out of scope)	PARTIAL

Failure Type	Detection	Recovery Action	In Scope?
GMS crash	gRPC deadline	Workers fall back to local-only; GMS recovers via Raft	YES
Network partition	gRPC stream lost	Workers operate local-only; GMS preserves registration 60s	PARTIAL

6.2 Failover Sequence

```
Time 0:    Worker A holds seq_1, epoch=3, committed_frontier=block_7
           Worker A is actively decoding, writing block_8 (PENDING)

Time 0+:   Worker A crashes (network failure, OOM, process kill)

Time 10s:  GMS heartbeat timeout for Worker A expires
           GMS marks Worker A as FAILED
           GMS initiates ownership transfer for all sequences owned by A:
             seq_1.owner = Worker_B
             seq_1.epoch = 4   (incremented)
             seq_1.block_8.state = PENDING -> discarded

Time 10s+: Worker B receives ownership notification
           Worker B queries GMS: get_committed_blocks(seq_1)
           GMS returns: blocks 0..7 (all COMMITTED), locations in MinIO

Time ~11s: Worker B loads blocks 0..7 from MinIO via S3 GET
           Worker B recomputes block_8 (the one PENDING block)
           Worker B resumes decode from block_9 onward

Time ?:    Worker A revives, tries to write block_8 with epoch=3
           GMS: 3 < current epoch 4 -> REJECT
           Worker A logs epoch mismatch, stops operating on seq_1
```

6.3 GMS High Availability via Raft

The GMS itself is replicated across three nodes using Raft consensus. This ensures that the metadata layer (Block Location Index, Ownership Registry) survives individual GMS node failures without data loss.

- Recommended implementation: use etcd as the Raft backend, exposing a thin gRPC service on top
- All GMS state is stored in etcd; GMS process is stateless and can be restarted freely
- Alternative: implement Raft directly using the raft-python or dragonboat (Go) library

```
# etcd-backed GMS pattern
# GMS reads/writes all state through etcd transactions
```

```
# Atomic ownership transfer (prevents race conditions):
etcd.transaction(
    compare=[etcd.Compare(etcd.Value(f'/seq/{seq_id}/epoch'), '=',
str(old_epoch))],
    success=[
        etcd.Put(f'/seq/{seq_id}/owner', new_worker_id),
        etcd.Put(f'/seq/{seq_id}/epoch', str(old_epoch + 1)),
    ],
    failure=[]  # CAS failed: another GMS node already transferred ownership
)
```

7. Implementation Roadmap

Ordered by priority. Each phase builds on the previous and produces a testable milestone.

Phase	Scope	Deliverable	Est. Effort
1: GMS Core	Block Location Index, MetadataClient stub, basic lookup/register RPCs, heartbeat	Workers can register blocks and query locations; GMS detects failures	1-2 weeks
2: Ownership + 2PC	Ownership Registry, epoch fencing, two-phase commit, failover trigger	End-to-end failover: worker crash -> new owner loads from MinIO	1-2 weeks
3: Deduplicatio n	Dedup check on write path, refcount management, PYTHONHASHSEED enforcement	Single copy per unique prefix across all workers	1 week
4: Admission Policy	Count-Min Sketch, cost model, workload classifier, access reporting	Remote write I/O reduced; low-reuse blocks skipped	1-2 weeks
5: GMS HA (Raft)	etcd backend, multi-node GMS, CAS ownership transfer	GMS survives single node failure	1-2 weeks
6: Validation	Fault injection harness, TTFT benchmarks, storage footprint measurement	Quantitative results for report	1 week

8. GMS Wire Protocol

Six core RPCs between MetadataClient and GMS. Initial implementation uses ZMQ (extending LMCache's existing controller transport). Production version uses gRPC unary RPCs.

RPC	Direction	Latency Target	Frequency
Lookup	Client -> GMS	< 5 ms	Per inference request
CheckAdmit	Client -> GMS	< 2 ms	Per KV chunk produced
RegisterWrite	Client -> GMS	< 1 ms	Per admitted chunk (Phase 1)
CommitWrite	Client -> GMS	< 1 ms	Per admitted chunk (Phase 2)
ReportAccess	Client -> GMS	< 1 ms	Batched every 100ms
Heartbeat	Client -> GMS	< 1 ms	Every 5s
GetCommittedBlocks	Client -> GMS	< 10 ms	On failover recovery
TransferOwnership	GMS internal	< 5 ms	On worker failure detection

9. Validation Plan

9.1 Fault Injection Tests

- Worker crash: kill -9 a vLLM process; verify new owner recovers within 15s with correct cache
- Split brain: revive killed worker; verify epoch fencing rejects its writes
- Write uncertainty: kill worker between register_write and commit_write; verify new owner discards PENDING block and recomputes
- GMS crash: kill one GMS node; verify Raft elects new leader and service resumes

9.2 Performance Benchmarks

Metric	Baseline (no GMS)	Target
TTFT after worker recovery	Full prefill (~1.6s per chunk)	<= 0.35s (MinIO load)
GMS lookup latency	N/A	< 5ms at 10K blocks
Remote write I/O	100% (spray-and-pray)	25-50% (admission gating)
Block copies per chunk	3-5 (LMCache default)	1-2 (dedup)
Failover detection time	N/A (no recovery)	<= 10s (2x heartbeat)

10. Key Design Decisions & Rationale

Decision	Alternative Considered	Rationale
GMS never touches KV bytes	GMS as data proxy	Avoids GMS becoming a throughput bottleneck; matches GFS design principle
Per-sequence ownership (not per-block)	Per-block ownership	Decode is sequential per sequence; per-sequence is sufficient and simpler
Epoch fencing at GMS level	Fencing at MinIO level	MinIO has no epoch semantics; GMS is the natural enforcement point
etcd for Raft backend	Implement Raft from scratch	etcd is battle-tested; avoids reimplementing consensus protocol
PYTHONHASHSEED fixed across workers	SHA-256 for all hashes	LMCache already uses prefix hash; fixing seed is the minimal change for deduplication
Heartbeat timeout = 10s	Shorter (5s) or longer (30s)	Balances false-positive failure detection against recovery latency
At most 1 uncommitted block invariant	Larger uncommitted window	Bounds recovery cost to recompute at most 1 block on failover

Appendix: Relationship to PESTO

This system implements the fault tolerance layer that PESTO (Section 7) explicitly leaves out of scope. PESTO's GMS provides the block location index, admission policy, and deduplication mechanisms. This work extends the GMS with:

- Ownership Registry: per-sequence owner_worker_id and epoch fields added to BlockMetadata
- Two-phase commit: PENDING/COMMITTED state machine on every write
- Epoch fencing: ownership transfer increments epoch; stale writes rejected
- Committed frontier: per-sequence tracking of last safely persisted block
- Failover recovery RPC: GetCommittedBlocks for new owner to reload cache

The PESTO admission policy (cost model, Count-Min Sketch, workload classifier) and deduplication logic are adopted as-is and are orthogonal to the fault tolerance additions. PESTO's GMS is therefore a strict subset of this system's GMS.